

Programação III (PG3)

Semestre de Inverno de 2026-2027

1º Trabalho prático

Data de Entrega: 30 de abril de 2027

OBJETIVOS: Implementar aplicações simples usando o paradigma da Programação Orientada por Objetos.

NOTA: a entrega do trabalho tem de incluir todo o código desenvolvido, incluindo os testes unitários que permitem validar a correção dos métodos e classes realizadas.

O trabalho está organizado em 3 grupos, que devem ser realizados pela ordem indicada. Os grupos são independentes entre si, pelo que a implementação de um grupo não depende da implementação dos restantes.

Grupo 1

1. Considere a definição da classe `Rect` que representa um retângulo, caracterizado por uma largura (`w`) e uma altura (`h`):

1. Indique o resultado da execução do método `main`, justificando o resultado de cada um dos `System.out`.
2. Adicione o que for necessário à classe `Rect` para que o resultado da execução do método `main` seja o seguinte:

```
> java i2627.trab1.grupo1.Rect
3 x 4
3 x 4
false
true
true
```

```
package i2627.trab1.grupo1;
public class Rect {
    private final double w;
    private final double h;
    public Rect(double w, double h) {
        this.w = w; this.h = h;
    }
    public static void main(String[] args){
        Rect r1 = new Rect(3, 4);
        Rect r2 = new Rect(3, 4);
        Rect r3 = r1;
        System.out.println( r1.toString() );
        System.out.println( r1 );
        System.out.println( r1 == r2 );
        System.out.println( r1 == r3 );
        System.out.println( r1.equals(r2) );
    }
}
```

3. Justifique os dois primeiros `System.out` do método `main` produzirem o mesmo resultado no *standard output*.

2. Adicione ou altere os membros na classe `Rect`, considerando os seguintes requisitos:

1. As coordenadas de origem `x` e `y`, igualmente do tipo `double`, representam a posição do canto superior esquerdo num plano;
2. O construtor com 4 parâmetros, com a ordem `x`, `y`, `w`, e `h`, estabelece a origem, a largura e a altura;
3. O construtor com dois parâmetros (`w` e `h`) considera a origem do retângulo no ponto 0,0. A implementação deste construtor deve chamar o construtor com quatro parâmetros;
4. As coordenadas `x` e `y` podem corresponder a valores negativos mas considere que `w` e `h` são sempre positivos.
5. O número de instâncias criadas com o construtor de dois parâmetros é contabilizado e o método `getNoOriginCounter` retorna o valor desse contador;
6. Os métodos `getX`, `getY`, `getW` e `getH` são métodos *getters/accessors*;
7. O método `toString` retorna uma `String` com o formato: `(x1,y1)-(x2,y2)`. Os pares `x1`, `y1` e `x2`, `y2` identificam as coordenadas do canto superior esquerdo e do canto inferior direito, respetivamente. Os valores devem ser apresentados com exatamente uma casa decimal;
8. O método de instância `equals` retorna `true` se os dois retângulos tiverem a mesma origem, largura e altura. Considere dois retângulos iguais se a diferença entre respetivas componentes não for superior a 0.01;
9. O método de instância `isOverlapped` recebe como parâmetro a referência para outro retângulo e retorna `true` se os dois retângulos estiverem totalmente ou parcialmente sobrepostos;
10. O método estático `parseRect` recebe a descrição de um retângulo como parâmetro, do tipo `String`, e retorna uma instância de `Rect`. O formato da descrição é igual ao formato produzido pelo método `toString`. No caso de formatação inválida, o método retorna `null`.

Nota: sugere-se a utilização dos métodos de instância da classe `java.lang.String`:

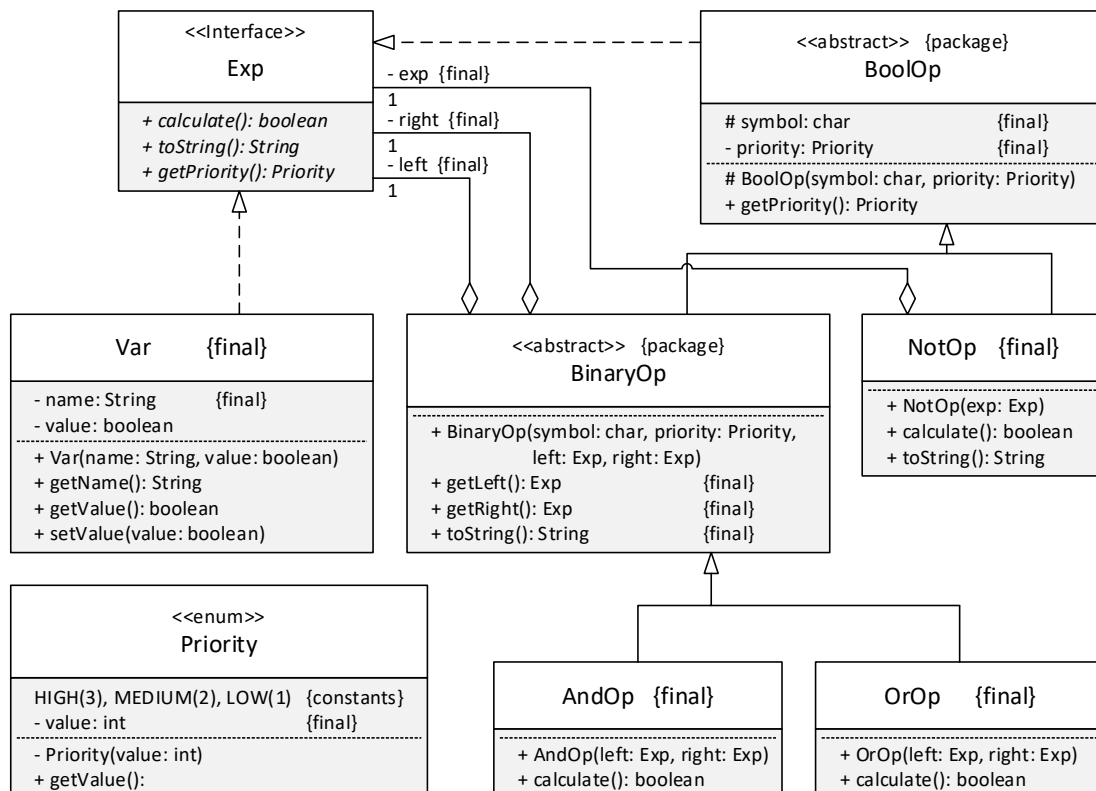
`String[] split( String regex )` – para individualizar partes de uma *string*;

`String substring( int beginIndex, int endIndex )` – para extrair uma *substring*;

`String substring( int beginIndex )` – para extrair uma *substring*.

Grupo2

Pretende-se implementar um sistema que avalia expressões booleanas baseadas nas operações *AND*, *OR* e *NOT*. Para tal, considere o diagrama de classes em baixo. As classes *AndOp*, *OrOp*, e *NotOp* modelam as operações booleanas e a classe *Var* modela uma variável da expressão. Qualquer uma destas operações, incluindo uma variável, têm associada uma prioridade, sendo que variáveis e operações *Not* têm a prioridade mais alta (*HIGH*), operações *AND* têm prioridade média (*MEDIUM*) e operações *OR* têm a prioridade mais baixa (*LOW*). O enumerado *Priority* define as prioridades através de três constantes: *HIGH*; *MEDIUM*; e *LOW*. Uma expressão está associada ou a uma variável ou a uma operação e respetivos operandos. Um operando é outra expressão. A interface *Exp* define as operações que qualquer expressão deve implementar, nomeadamente, o método *calculate* produz o resultado da expressão, o método *toString* retorna a representação da expressão na forma de uma *string* e o método *getPriority* retorna a prioridade da expressão. Chama-se a atenção que a representação da expressão numa *string* pode envolver a utilização de parêntesis, de forma a respeitar a ordem de execução dos termos da expressão. No diagrama de classes, a característica *{final}* aplicada a uma classe significa que não se pode derivar da classe, aplicado a um campo da classe, significa que o campo não pode ser reatribuído depois de iniciado e aplicado a um método, significa que o método não pode ser sobreposto. A característica *{package}* significa que a classe tem visibilidade apenas dentro do *package*.



1. Considerando o diagrama estático de classes:

1. Defina a interface *Exp* que reúne as operações que qualquer expressão tem de suportar.
2. Defina o enumerado *Priority* com as constantes *HIGH*, *MEDIUM* e *LOW* com os valores 3, 2, e 1, respetivamente. O método *getValue* retorna o valor associado à constante.
3. Defina a classe *Var*, caracterizada por um nome (*name*) e por um valor (*value*) e cumprindo os requisitos:
 - O construtor recebe os valores iniciais de *name* e *value*;
 - Os restantes métodos são métodos *accessor* e *mutator*.
4. Defina a classe *BoolOp* que representa a classe base das operações booleanas. Reúne o símbolo da operação booleana (*symbol*) na forma de um caractere, e a prioridade (*priority*). Estas características são estabelecidas no construtor.
5. Defina a classe *NotOp*. O construtor desta classe recebe como parâmetro a expressão a negar e o símbolo desta operação é o caractere `'!'`.
6. Defina a classe *BinaryOp* que representa a classe base das operações booleanas binárias. Deve cumprir os seguintes requisitos:

- O construtor recebe como parâmetros o símbolo e a prioridade da operação, e as expressões correspondentes aos termos da esquerda e da direita do operador;
- Define dois métodos *accessor*, um para cada expressão;
- Define o método `toString` que retorna uma *string* com a representação da expressão. Chama-se a atenção que este método não pode ser sobreposto em classes derivadas.

7. Defina as classes `AndOp` e `OrOp`, de acordo com os seguintes requisitos:

- O construtor recebe como parâmetros as duas expressões correspondentes aos termos da operação;
- Os caracteres `'&'` e `'|'` são os símbolos que representam as operações *AND* e *OR*, respetivamente.

Nota: os exemplos seguintes ilustram representações de expressões. De realçar que a utilização de parêntesis é usada apenas quando necessário para garantir a ordem das operações, considerando as prioridades dos operadores:

```
A
!A
A | B & C
(A | B) & C
!(A | !B)
A & B & C
A | B | C
```

2. Adicione aos testes unitários deste grupo um teste unitário com os seguintes requisitos:

1. Implemente a expressão `!((A | B & C) & (!A & B | !C))`;
2. Valide o retorno da representação da expressão na forma de *string*;
3. Valide o resultado da expressão para todas as combinações entre A, B e C.

Nota: sugere-se a construção de um *array* de *records* Java. Considere o *record* `UnitTest` que representa uma das combinações a testar (A, B e C) e o resultado esperado (R):

```
record UnitTest(boolean A, boolean B, boolean C, boolean R) {}
UnitTest[] unitTests = {
    new UnitTest(false, false, false, <substituir por true ou false>),
    new UnitTest(false, false, true, <substituir por true ou false>),
    ...
};
```

Grupo3

Pretende-se implementar um sistema de gestão de frota de veículos.
Em construção...

Bom trabalho!